Week 2 Module

Searching – An Introduction

Searching is the Process of locating given value Position in a list of values, i.e search is a Proces of finding a value in a list of values. Search is said to be successful if the element being searched is found or unsccessful when the element being searched is not found. A search typically answer in either True or False as to whether the item is present.

Two techniques are used for searching:

1) Linear Search
2) Binary search

Linear Search

A linear Search is the basic and simple search algori
In linear Search, a sequential search is made ov
all element one by one. In other words linear search
Searches an element or value from an array
till the desired element is not found in sequential
order.

Data Requirement:– Works perfectly on both sorted and unsorted arrays.

Approach:– Iterative / sequential traversal

The time complexity of linear search O(n)

Multidimensional array can be used

It is less complex

Teacher's Signature .....

```
int linear search (int arr[], int size, int target)
{
  for (int i = 0; i < size; i++)
  { if (arr[i] == target)
    {
      return i;
    }
  }
    return -1;
}
int main()
{
  int arr[] = {20, 5, 8, 40, 12};
  int size = sizeof(arr) / sizeof(arr[0]);
  int target = 40;
  int result = linear seach(arr, size, target);
  if (result != -1)
    Printf("Element found at index: %d \n", result);
else
    printf("Element not found. \n");
  return 0;
}
```

Time Complexity:-

- Best Case - O(1) :- Occurs when the target element is at the very first position of the array.
- Worst-Case : O(n) :- Occurs when the target element is at the last position
- Average Case : O(n) :- Occurs when the element is located somewhere around the middle of the collection

Teacher's Signature .....

2) Binary Search

Binary search is a searching algorithm that operates on a sorted or monotonic search space, repeatedly dividing it into halves to find a target value or optimal answer in logarithmic time O(logN)

Ex:—

```
#include <stdio.h>
int binarySearch(int arr[], int left, int right,
                 int key)
{
  // are no elements to consider in the
     given subarray
  while (left <= right)
  {
  // calculating mid point
    int mid = left + (right - left) / 2;

// check if key is present at mid
  if (arr[mid] == key)
  { return mid;
  }
// if key greater than arr[mid], ignore
   left half
  if (arr[mid] < key)
{
  left = mid + 1;
}
```

Teacher's Signature .....

```
// If key is smaller than or equal to arr[mid],
// Ignore right half
  else
  {
    right = mid-1;
  }
}
// If we reach here, then element was not
Present
  return -1;
}
int main()
{
  int arr[] = { 2, 5, 8, 12, 16, 23, 38, 56, 72, 91};
  int size = sizeof(arr) / sizeof(arr[0]);
    // Element to be searched
    int key = 23
    int result =
    binarySearch(arr, 0, size-1, key);
    if (result == -1)
    {
      Printf("Element is not Present in array");
    }
    else
    {
      Printf("Element is Present at index %d",
        result);
    }
    return 0;
}
```

Output

Element is present at index 5;

Teacher's Signature .....

Time complexity : O(log N) ; where N is the number of elements in the array

Auxiliary space: O(log N) ; where N is the number of elements in the array.

Sorting

→ The process of arranging the elements in a list in either ascending or descending order.

→ Types of Sorting :-

1) Bubble Sort
2) Insertion Sort
3) Selection Sort

Bubble Sorting

→ Bubble sort is the easiest sorting algorithm to implement.

→ It is inspired by observing the behaviour of air bubbles over foam.

→ It is an in-place sorting algorithm.

→ It uses no auxiliary data structures (extra space) while sorting.

How Bubble Sort Works

→ Bubble sort uses multiple passes (scans) through an array.

→ In each pass, bubble sort compares the

Teacher's Signature .....

adjacent elements of the array.

→ It then swaps the two elements if they are in the ~~working~~ wrong order.

→ In each pass, bubble sort places the next largest element to its proper position.

→ In short, it bubbles down the largest element to its correct position.

Time Complexity

Bubble Sort Algorithm	Time complexity
Best Case	$O(n)$
Average case	$O(n^2)$
Worst case	$O(n^2)$

Ex:-
```
#include <stdio.h>
Void swap (int *arr, int i, int J)
{
    int temp = arr[i];
    arr[i] = arr[J];
    arr[J] = temp;
}
Void bubble sort (int arr[], int n)
{
    for (int i = 0; i < n-1; i++)
    {
        for (int j = 0; j < n-i-1; J++)
        {
            if (arr[J] > arr[J+1])
```

Teacher's Signature .....

```
            swap(arr, j, j+1);
        }
    }
}

int main()
{
    int arr[] = {5, 6, 1, 3};
    int n = sizeof(arr) / sizeof(arr[0]);
    // Calling bubbleSort on array arr
    bubbleSort(arr, n);
    for(int i=0; i<n; i++)
    Printf("%d", arr[i]);
    return 0;
}
```

Insertion Sort

→ Insertion Sort is a simple, comparison-based sorting algorithm that works by building a sorted array one element at a time.

→ It conceptually divides the array into a sorted and unsorted part.

Q:-

```
#include <stdio.h>
void insertionSort(int arr[], int n)
{
  int i, key, j;
  for(i = 1; i < n; i++)
  {
    key = arr[i];
    j = i - 1;
    while (j >= 0 && arr[j] > key)
    {
      arr[j+1] = arr[j];
      j = j - 1;
    }
    arr[j+1] = key;
  }
}
void printArray(int arr[], int n)
{
  for(int i=0; i<n; i++)
  {
    printf("%d", arr[i]);
  }
  printf("\n");
}
int main()
{
  int data[] = {12, 11, 13, 5, 6};
  int size = sizeof(data) / sizeof(data[0]);
  Print("Original array: \n");
  printArray(data, size);
  insertionSort(data, size);
  print("Sorted array in ascending orda: \n");
  printArray(data, size);
}
```

Teacher's Signature .....

Time Complexity:

- Best case: $O(n)$ - Occurs when the array is already sorted. The inner loop never executes
- Average case: $O(n^2)$:- Triggers for random order allocations.
- Worst case: $O(n^2)$:- Triggers when the array is reverse sorted.
- Space complexity: $O(1)$:- In-place algorithm: only auxiliary variables (key, i, j) are used

Selection Sort

The selection sort is a simple comparison based sorting algorithm that sorts a collection by repeatedly finding the minimum element and placing it in its correct position in the list. It is very simple to implement and is preferred when you have to manually implement the sorting algorithm for a small amount of datasets.

Selection Sort Algorithm in C

- Start with the entire list as unsorted.
- Start traversing the list from the first element
- For each element:
  - Find the smallest element from the current position to the end of the list
  - Swap it with current element to place it in its correct position

Teacher's Signature .....

- Move to the next element and repeat this for all elements until the array is sorted.

```
#include <stdio.h>
void selectionSort(int arr[], int N)
{
    for (int i = 0; i < N-1; i++)
    {
        int min_idx = i;
        for (int j = i+1; j < N; j++)
        {
            if (arr[j] < arr[min_idx])
            {
                min_idx = j;
            }
        }
        int temp = arr[min_idx];
        arr[min_idx] = arr[i];
        arr[i] = temp;
    }
}

int main()
{
    int arr[] = {64, 25, 12, 22, 11};
    int N = sizeof(arr) / sizeof(arr[0]);
    printf("Unsorted array: \n");
    for (int i = 0; i < N; i++)
    {
        printf("%d", arr[i]);
    }
    printf("\n");
```

Teacher's Signature .....

```
selectionSort(arr, N);
printf("Sorted array: \n");
for(int i=0; i<N; i++)
{
printf("%d", arr[i]);
}
printf("\n");
}
```

Output

Unsorted array: 64 25 12 22 11

Sorted array: 11 12 22 25 64

1.) Find Student Roll Number (Linear Search)

Problem statement :—

Your class teacher has a list of student roll numbers sorted in the order they registered. Since the list is unsorted, you need to find whether a particular student roll number exists.

Write a Program to search for the given roll number using linear search.

```
#include<stdio.h>
int main()
{
    int n, i, searchRoll, found = 0;
// 1. Get the total number of students
    printf("Enter the number of students registered:");
    scanf("%d", &n);
    int rollNumbers[n];

// 2. Input the roll numbers
    printf("Enter the roll numbers of the students:\n");
    for(i=0; i<n; i++)
    {
        printf("student %d : ", i+1);
        scanf("%d", &rollNumbers[i]);
    }
```

```
// 3. Get the roll number to search for
printf("\nEnter the roll number you want to
        search for : ");
scanf("%d", &SearchRoll);

// 4. Perform linear search
for(i=0; i<n; i++)
{
  if(rollNumber[i] == SearchRoll)
    printf("\n Student found! Roll numb
            %d is at position %d
    (Index %d).\n", SearchRoll, i+1
    found = 1;
    break;
  }
}

// 5. If the roll number was not in the
   list
if(!found)
{
 Printf("\n Student with roll number %d
     does not exist in the registration
     list.\n", SearchRoll);
}
  return 0;
}
```

Output

Enter the no of students registered : 5
Enter the roll number of the student :
Student 1 : 105
Student 2 : 102
Student 3 : 108
Student 4 : 101
Student 5 : 104

Enter the roll number you want to search for : 101
Student found Roll number 101 is at position 4 (Index 3)

2.) Search Product ID (Binary search)

Problem Statement :-
An online store keeps product IDs sorted in ascending order. A customer wants to know whether a particular product exist in inventory.

Write a program to search the product using Binary search.

```
#include <stdio.h>
// Function to Perform Binary Search
int binary search (int arr[], int size, int target)
{
    int left = 0;
    int right = size - 1;
```

```
while (left <= right)
{
    int mid = left + (right - left) / 2;
    // Check if target is present at mid
    if (arr[mid] == target)
    {
        return mid; // Product found, return its index
    }
    // If target is greater, ignore left half
    if (arr[mid] < target)
    {
        left = mid + 1;
    }
    // If target is smaller, ignore right half
    else
    {
        right = mid - 1;
    }
}
return -1;
}

int main()
{
    // Sample sorted inventory of Product IDs
    int inventory[] = {101, 204, 305, 412, 559,
                       629, 789};
    int size = sizeof(inventory) / sizeof
               (inventory[0]);
```

```
in searchID);
printf("Enter the product ID to search: ");
scanf("%d", &searchID);

// Call the binary search function
int result = binarySearch(inventory, size, searchID);

// Display the result
if (result != -1)
{
    printf("Product ID %d exists in inventory
            at index %d.\n",
            searchID, result);
}
else
{
    printf("Product ID %d does not exist in
            inventory.\n", searchID);
}
return 0;
}
```

Output

```
Enter the Product ID to search: 412
Product ID 412 exists in inventory.
```

3. Arrange Exam Scores (Bubble sort)

Problem statement :- A teacher wants to arrange students exam score in ascending order to Prepare the result sheet

Teacher's Signature .....

Write a Program using Bubble Sort.

```
#include <stdio.h>
int main()
{
  int n, i, j, temp;
// Ask the teacher for the number of students
Printf("Enter the number of students: ");
scanf("%d", &n);
int scores[n];
// Input the exam scores
  printf("Enter %d exam scores:\n", n);
  for (i=0; i<n; i++)
{ printf("student %d score:", i+1);
  scanf("%d", &scores[i]);
}
// Bubble Sort Algorithm (Ascending Order)
  for (i=0; i<n-1; i++)
{ for (j=0; j<n-i-1; j++)
{
// Swap if the current score is greater than the
    next score
  if (scores[j] > scores[j+1]
  {
    temp = scores[j];
    scores[j] = scores[j+1];
    scores[j+1] = temp;
}}}
```

```
// Display the sorted result
printf("\n --- Result sheet (Ascending order) ---\n");
for (i = 0; i < n; i++)
{ printf("score: %d\n", scores[i]);
} return 0;
}
```

Output

```
Enter the no. of students: 5
Enter 5 exam scores:
Student 1 score: 85
Student 2 score: 62
Student 3 score: 95
Student 4 score: 45
Student 5 score: 78

--- Result sheet (Ascending order) ---
score: 45
score: 62
score: 78
score: 85
score: 95
```

4) Arrange library Book Numbers (Insertion sort)

Problem statement

A librarian recives book serial numbers one by one and want to place each new book in the correct position.

Write a Program using Insertion Sort to arrange book numbers

```
#include<stdio.h>
// Function to Perform insertion Sort
void insertionSort(int books[], int n)
{
    int i, key, j;
    for(i=1; i<n; i++)
    {
        key = books[i]; // The new book received by the Librarian
        j = i-1;
        /* Move elements of books[0..i-1]
that are greater than key to one Position
ahead of their current Position*/
        while(j>=0 && books[j]>key)
        {
            books[j+1] = books[j];
            j = j-1;
        }
        // Insert the book into its correct sorted
        Position
        books[j+1] = key;
    }
}
// Function to print the array of books
void printBooks(int books[], int n)
{
    for(int i=0; i<n; i++)
```

Teacher's Signature .....

```
{
    printf("%d", books[i]);
}
    printf("\n");
}

int main()
{
    int num_books;
    printf("Enter the number of books to sort:");
    scanf("%d", &num_books);
    int books[num_books];
    printf("Enter the book numbers one by
            one:\n");
    for(int i = 0; i < num_books; i++)
    {
        printf("Book %d", i+1);
        scanf("%d", &books[i]);
    }
    printf("\n -- Unsorted shelf --\n");
    printBooks(books, num_books);
// sorting the books using Insertion sort
    insertionSort(books, num_books);
    printf("\n -- Correctly Sorted shelf --\n");
    printBooks(books, num_books);
    return 0;
}
```

Output

Enter the no. of books to sort : 5
Enter the book numbers one by one:
Book 1 : 105
Book 2 : 82
Book 3 : 201
Book 4 : 15
Book 5 : 94

Unsorted Shelf
105 82 201 15 94

Correctly Sorted Shelf
15 82 94 105 201

5.) Rank Players by Score (selection sort)

Problem statement :—
A gaming club want to rank players based on scores from lowest to highest write a program using selection sort

```
#include <stdio.h>
#include <string.h>
// Define a structure to hold Player data
Struct Player
{
```

```
    char name[50];
    int score;
};

// Function to perform selection sort
void selectionSort(struct Player arr[], int n)
{
    int i, j, min_idx;
    struct Player temp;
    // Move boundary of unsorted subarray one by one
    for (i = 0; i < n-1; i++)
    {
        // Find the minimum element in unsorted array
        min_idx = i;
        for (j = i+1; j < n; j++)
        {
            if (arr[j].score < arr[min_idx].score)
            {
                min_idx = j;
            }
        }
    }
    // Swap the found minimum element with the
    first element
    if (min_idx != i)
    {
        temp = arr[i];
        arr[i] = arr[min_idx];
        arr[min_idx] = temp;
    }
}
```

Teacher's Signature .....

```
int main()
{
    int n, i;
    printf("Enter the number of players: ");
    scanf("%d", &n);

    Struct Player players[n];

    // Input player details
    for(i=0; i<n; i++)
    {
        printf("\n Enter details for player %d: \n",
                                        i+1);
        printf("Name: ");
        scanf("%s", players[i], name);
        printf("Score: ");
        scanf("%d", & Players[i], score);
    }
    // Sort the Players based one scores (lowest
                                        to highest)
    selectionSort(Players, n);

    // Display the leaderboard
    printf("\n --- leaderboard (lowest to highest
                                    Score ---\n");
    Printf("%-5s %-20s %-10s\n", "Rank",
                                "Player Name", "Score");

    return 0;
}
```

Output

Enter the number of players : 4
Enter details for player 1:
Name : Alex
Score : 450

Enter details for player 2 :
Name : Sam
Score : 120

Enter details for player 3 :
Name : Jordan
Score : 310

Enter details for player 4 :
Name : Taylor
score : 95

leaderboard (lowest to Highest score)

Rank	Player Name	Score
1	Taylor	95
2	Sam	120
3	Jordan	310
4	Alex	450

Week 2 Module

Searching - An Introduction

Searching is the process of locating given value Position in a list of values, i.e search is a Process of finding a value in a list of values. Search is said to be successful if the element being searched is found or unsuccessful when the element being searched is not found. A search typically answer is either True or False as to whether the item is present.

Two techniques are used for searching:

1) Linear Search
2) Binary Search

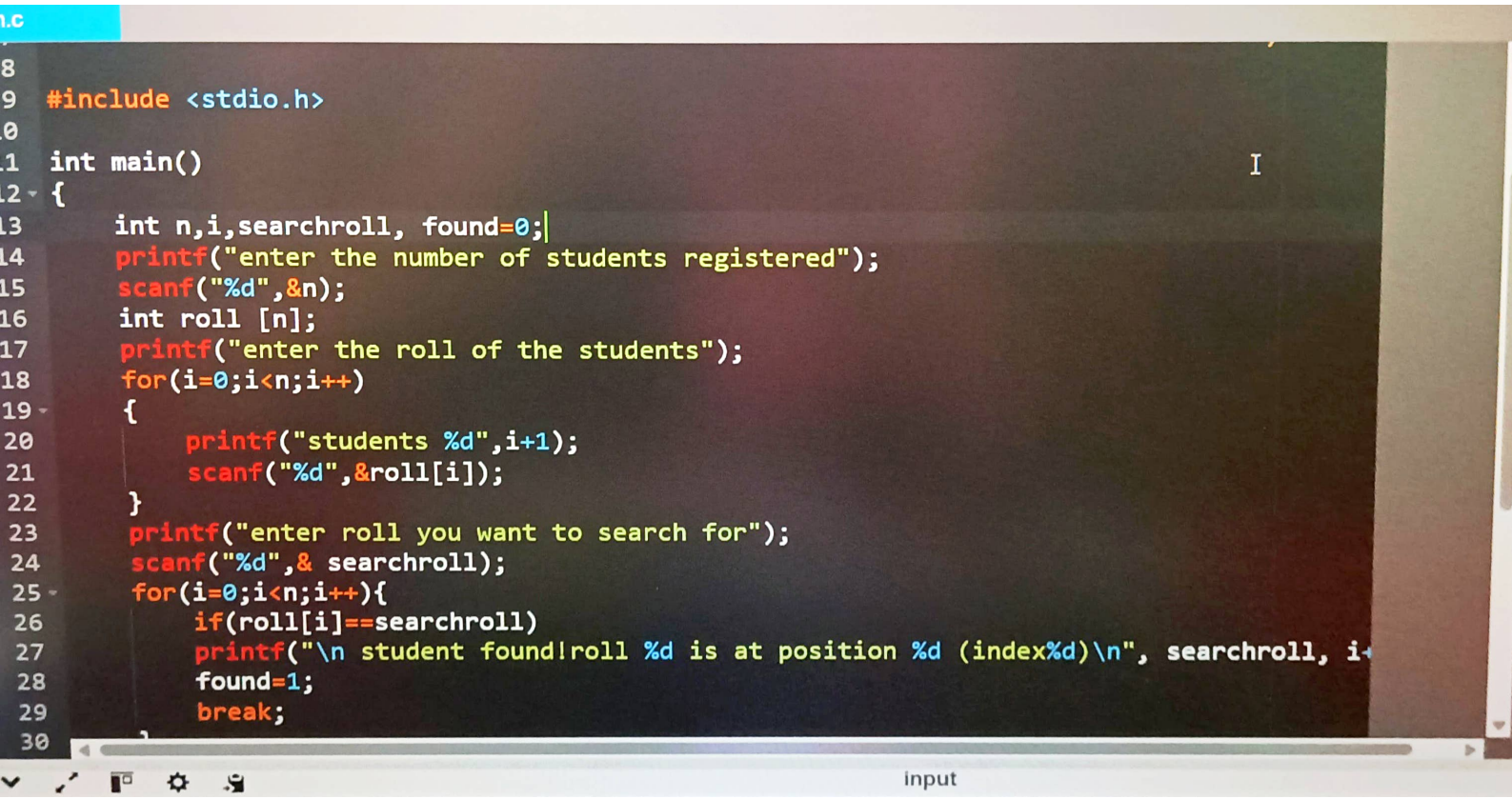
1) Linear Search

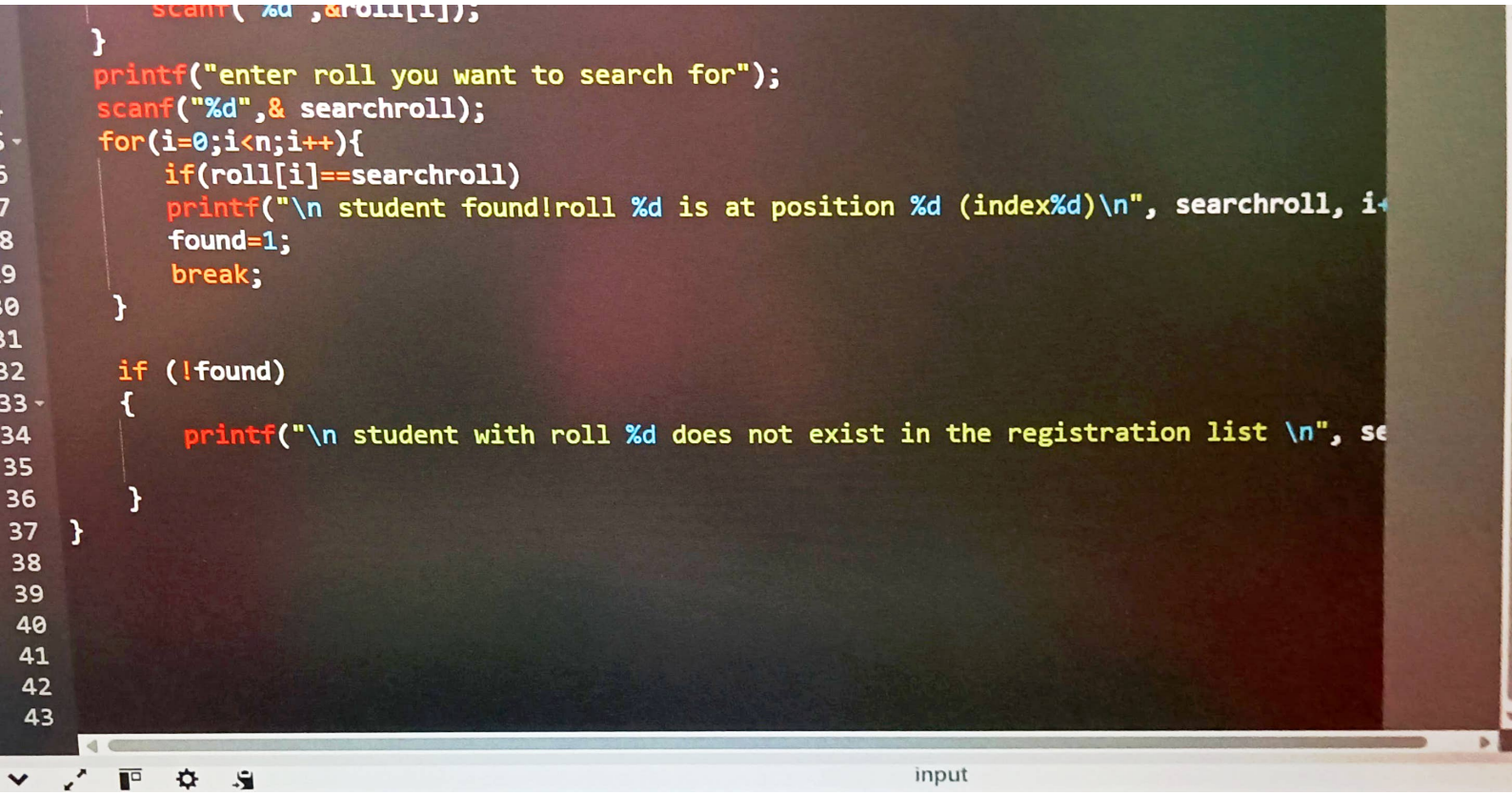
A linear Search is the basic and simple search algorith. In linear Search, a sequential search is made over all element one by one. In other words linear search searches an element or value from an array till the desired element is not found in sequential order.

- Data Requirement:- Works perfectly on both sorted and unsorted arrays.
- Approach:- Iterative / sequential traversal

→ The time complexity of linear search O(n)
→ Multidimensional array can be used
→ It is less complex

Teacher's Signature .....


n.c
#include <stdio.h>
int main()
{
int n,i,searchroll, found=0;
printf("enter the number of students registered");
scanf("%d",&n);
int roll [n];
printf("enter the roll of the students");
for(i=0;i<n;i++)
{
printf("students %d",i+1);
scanf("%d",&roll[i]);
}
printf("enter roll you want to search for");
scanf("%d",& searchroll);
for(i=0;i<n;i++){
if(roll[i]==searchroll)
printf("\n student found!roll %d is at position %d (index%d)\n", searchroll, i
found=1;
break;
input


scanf("%d",&roll[i]);
}
printf("enter roll you want to search for");
scanf("%d",& searchroll);
for(i=0;i<n;i++){
if(roll[i]==searchroll)
printf("\n student found!roll %d is at position %d (index%d)\n", searchroll, i
found=1;
break;
}
if (!found)
{
printf("\n student with roll %d does not exist in the registration list \n", se
}
}
input